Auditoría Web Backend IA

Agente conversacional + MCP + RAG

Laboratorio 4

Proyecto: Kohem Chemicals

Campo	Valor
Estudiante	Nombre del Estudiante
Materia	Calidad de Software y Auditoría de Sistemas
Programa	Tecnología de Desarrollo de Software
Institución	Institución Universitaria Pascual Bravo
Versión auditada	Kohem Chemicals MVP · rama lab4-mcp-rag
Fecha	10/11/2026

Documento adaptado del laboratorio original de calculadora FastAPI al backend de Kohem Chemicals. Conserva los diez casos P01–P10 y sustituye los componentes matemáticos por las reglas de negocio, el adaptador RAG y el servidor MCP del MVP.

1. Descripción del proyecto

Kohem Chemicals es un Sistema de Información Transaccional para una distribuidora de materias primas: catálogo e inventario, registro de pedidos, asesoría con agente conversacional y auditoría de operaciones. El MVP está construido con FastAPI, SQLAlchemy asíncrono y SQLite en el backend, y React con Vite en el frontend.

El backend incorpora un asistente que recibe un mensaje en lenguaje natural, recupera políticas comerciales mediante un adaptador RAG, invoca herramientas de catálogo, stock y cotización a través de un servidor MCP real y devuelve una respuesta estructurada. La arquitectura es Onion: dominio, aplicación, infraestructura y presentación.

La auditoría se concentra en la corrección de los cálculos de pedido, la validación de solicitudes, el contrato HTTP, la integración entre capas, el comportamiento del adaptador MCP y la aplicación de las reglas RAG. No se realizan pruebas destructivas ni de carga.

Limitaciones declaradas de la versión auditada

- El asistente funciona sin modelo externo por defecto: la intención se identifica con reglas léxicas y expresiones regulares. Si se configura DEEPSEEK_API_KEY, un modelo de lenguaje interpreta la consulta; con AI_MODE=conversacional también redacta la respuesta usando las cifras calculadas por el dominio.
- El RAG es un adaptador simulado en memoria que recupera reglas por palabras clave, no una base vectorial con búsqueda semántica.
- El asistente cotiza y consulta stock, pero no registra pedidos ni consulta por identificador.

Estas características deben registrarse como limitaciones funcionales de la versión auditada, no como fallos de un LLM real.

2. Objetivo general

Ejecutar pruebas funcionales y de calidad sobre el backend de Kohem Chemicals, verificando la corrección de las reglas de negocio y la integración entre presentación, aplicación, dominio, RAG y MCP.

3. Objetivos específicos

- Verificar los cálculos de subtotal, IVA, tope comercial y descuento de stock.
- Evaluar las validaciones de entrada y el tratamiento de errores, incluido el contraste con los endpoints legados.
- Comprobar el contrato de la API mediante Swagger y solicitudes HTTP.
- Verificar la comunicación entre el caso de uso y el adaptador MCP, incluido el transporte stdio.
- Evaluar la recuperación y aplicación de reglas del adaptador RAG.
- Comprobar la separación de responsabilidades entre las capas Onion.

- Ejecutar las pruebas automatizadas existentes y documentar hallazgos y correcciones.

4. Alcance

Se auditan los siguientes archivos del MVP, ubicados en «1. Diagnostico Auditoria/MVP»:

Archivo	Capa	Qué se audita
app/domain/services.py	Dominio	Subtotal, IVA, tope comercial y stock mínimo.
app/domain/entities.py	Dominio	Entidades puras, sin frameworks.
app/application/ports.py	Aplicación	Contratos IRagRepository e IMcpToolExecutor.
app/application/ai_use_cases.py	Aplicación	Orquestación RAG → MCP → respuesta.
app/infrastructure/mock_vector_db.py	Infraestructura	Recuperación de políticas por palabras clave.
app/infrastructure/mcp_server.py	Infraestructura	Herramientas MCP y transporte stdio.
app/infrastructure/deepseek_llm.py	Infraestructura	Motor de lenguaje opcional (DeepSeek).
app/infrastructure/repositories.py	Infraestructura	Acceso a SQLite.
app/presentation/routers/ai_router.py	Presentación	Contrato HTTP y validación del mensaje.
app/presentation/validacion.py	Presentación	Validación centralizada y errores 422/400.
app/presentation/dependencies.py	Presentación	Inyección de dependencias.
tests/ (5 archivos)	Pruebas	Suite automatizada del dominio, MCP, IA y HTTP.
docs/laboratorio4/	Evidencias	Scripts f01–f12, JSON y consolas.

Quedan fuera del alcance: el frontend React, la interfaz de usuario, las pruebas de carga y cualquier prueba destructiva. Los endpoints de materias primas y pedidos se usan solo como contraste del manejo de errores.

5. Arquitectura auditada

Capa	Responsabilidad	Depende de
Dominio	Reglas puras de precio, IVA, tope y stock.	Nada externo
Aplicación	Caso de uso que orquesta RAG y MCP.	Puertos (interfaces)
Infraestructura	RAG simulado, servidor MCP y repositorios SQLite.	Dominio y SDK de MCP
Presentación	Rutas FastAPI, validación y dependencias.	Aplicación

Flujo de una solicitud:

```
POST /api/v1/ai/chat
  → AIAssistantUseCase.respond()
      → IRagRepository.buscar_contexto()        (políticas comerciales)
      → ILLMProvider.interpretar()               (opcional: DeepSeek; si falla,
reglas)
```

```
      → IMcpToolExecutor.ejecutar_herramienta()  (listar_materiales /
consultar_stock / calcular_pedido)
          → MCPServer (protocolo MCP)
              → PricingDomainService + SQLiteRawMaterialRepository
  → AIChatResponseDTO { response, transfer_to_employee, reason, motor, datos }
```

5.1 Motor de inteligencia

El asistente tiene dos motores intercambiables y siempre deja constancia del usado en el campo «motor» de la respuesta:

- Reglas (por defecto): recuperación RAG por palabras clave, expresiones regulares para cantidades y unidades, y comparación difusa para nombres de materiales. No requiere red.
- DeepSeek (opcional): si DEEPSEEK_API_KEY está definida, el modelo interpreta la intención y las entidades y devuelve un JSON validado. Si la red falla, el JSON es inválido o la clave no existe, el caso de uso continúa con reglas.

Configuración (archivo .env, nunca versionado):

```
DEEPSEEK_API_KEY=sk-...
DEEPSEEK_BASE_URL=https://api.deepseek.com
DEEPSEEK_MODEL=deepseek-chat
LLM_TIMEOUT_SECONDS=20
AI_MODE=conversacional
AI_HISTORY_MAX=8
```

El adaptador está en app/infrastructure/deepseek_llm.py y usa el formato JSON de la API compatible con OpenAI. La clave no se registra en logs ni en evidencias.

5.2 Modo conversacional

El comportamiento se controla con la variable AI_MODE:

AI_MODE	Comportamiento	Uso recomendado
asistente (por defecto)	Plantilla fija + reglas o DeepSeek para la intención.	Auditoría P01–P11: respuestas reproducibles.
conversacional	DeepSeek redacta la respuesta con los datos calculados por el dominio.	Atención al cliente y P12.

En modo conversacional el modelo actúa como asesor comercial: recibe el catálogo real (materiales, categorías, precios y stock) y las políticas aplicables, además de un bloque «Datos verificados» con las cifras calculadas. Puede responder preguntas abiertas sobre qué vende la empresa, orientar al cliente y pedir la información que falte, sin inventar precios ni existencias. Si el modelo falla, se devuelve la plantilla del modo asistente.

El cliente puede enviar el historial para dar contexto:

```
{
  "message": "¿Y cuánto hay de ese material?",
  "historial": [
    {"role": "user", "content": "Quiero dos kilos de glicerina"},
```

```
      {"role": "assistant", "content": "..."}
    ]
  }
```

Para ejecutar f01–f11 el backend debe estar en AI_MODE=asistente. El caso f12 levanta su propia instancia en el puerto 8001 con AI_MODE=conversacional, de modo que la auditoría no depende del modo configurado en el servicio principal.

El servidor no guarda conversaciones: el historial lo envía el cliente y se recorta a AI_HISTORY_MAX mensajes. La respuesta incluye el campo «datos» con las cifras usadas.

6. Preparación del entorno

1. Abrir PowerShell en «1. Diagnostico Auditoria\MVP».

2. Instalar dependencias si hace falta (incluye mcp>=2.3.0):

```
.\venv\Scripts\python.exe -m pip install -r requirements.txt
```

3. Opcional: copiar .env.example a .env, completar DEEPSEEK_API_KEY y elegir AI_MODE (asistente o conversacional). Sin clave, el asistente usa el motor de reglas.

4. Iniciar el servicio:

```
.\venv\Scripts\python.exe -m uvicorn app.presentation.main:app --reload
```

5. Confirmar que el servidor escucha en http://127.0.0.1:8000 y abrir Swagger en http://127.0.0.1:8000/docs.

6. En otra terminal, ejecutar la suite:

```
.\venv\Scripts\python.exe -m pytest -q
```

7. Ejecutar los scripts de evidencia desde la carpeta docs\laboratorio4:

```
cd docs\laboratorio4
..\..\venv\Scripts\python.exe f01_operaciones.py
```

8. El servidor MCP también puede levantarse de forma autónoma por stdio (no es HTTP):

```
.\venv\Scripts\python.exe -m app.infrastructure.mcp_server
```

Credenciales de demostración

Rol	Usuario	Contraseña
admin	admin	AdminPass123!
cliente	cliente	ClientPass123!

El token dura 60 minutos y se envía como «Authorization: Bearer <token>».

7. Metodología

Se aplica el ciclo Planear–Hacer–Verificar–Actuar (PHVA):

Fase	Actividad en el laboratorio	Evidencia
Planear	Definir casos, datos de entrada y resultado esperado.	Esta guía, sección 8
Hacer	Ejecutar pytest, Swagger y los scripts f01–f12.	evidencias/*.json y *.txt

Fase	Actividad en el laboratorio	Evidencia
Verificar	Comparar lo obtenido con lo esperado y clasificar hallazgos.	RESULTADOS_LAB4.md
Actuar	Proponer correcciones y repetir las pruebas afectadas.	Informe final

8. Casos de auditoría y pruebas funcionales

P01 — Operaciones del dominio y cálculo de pedido

Componente: Dominio: app/domain/services.py (PricingDomainService) y herramienta MCP calcular_pedido; prueba tests/test_pricing_domain.py.

Procedimiento:

1. Ejecutar «pytest -q tests/test_pricing_domain.py».
2. Comprobar subtotal 18.000 × 2 = 36.000, total de dos ítems, IVA del 19% sobre 48.000 = 9.120 y redondeo a dos decimales.
3. Desde Swagger, enviar «Cotiza 2 kg de glicerina y 3 kg de ácido cítrico» y comparar el subtotal con los precios de GET /materials.
4. Ejecutar el script f01_operaciones.py y revisar f01_operaciones.json.

Resultado esperado: Las cuatro comprobaciones del dominio son correctas y la cotización coincide con el catálogo.

Evidencia: f01_operaciones.json · salida de pytest · captura de Swagger

P02 — Stock insuficiente y consistencia transaccional

Componente: Dominio y aplicación: calcular_pedido, OrderUseCases.create_order; pruebas tests/test_mcp_adapter.py.

Procedimiento:

1. Ejecutar «pytest -q tests/test_mcp_adapter.py».
2. Desde Swagger, cotizar 10.000 kg de glicerina y revisar el código HTTP y el detalle.
3. Crear un pedido con dos ítems donde el segundo supera el stock y comprobar que el stock del primer ítem no cambia.
4. Enviar un pedido con cantidad 0 y comprobar que devuelve 422.
5. Ejecutar f02_stock.py.

Resultado esperado: No se permite la sobreventa, un pedido fallido no reserva stock parcialmente y la cantidad 0 se rechaza con 422; ninguna de estas rutas produce HTTP 500.

Evidencia: f02_stock.json

P03 — Valores límite: negativos, decimales y redondeo

Componente: Dominio y validaciones: RawMaterialCreateDTO, OrderItemCreateDTO y PricingDomainService.

Procedimiento:

1. Verificar en el dominio: subtotal(0.1, 3) = 0.3, subtotal(1234.567, 3) = 3703.70 y subtotal(18000, 0.5) = 9000.0.
2. Crear una materia prima con price = -100 y otra con stock = -1; registrar el código HTTP.
3. Enviar un pedido con quantity = -2 y otro con quantity = 0.5.
4. Crear un material con precio 1234.567, pedir 3 unidades y comprobar el total 3703.70.
5. Ejecutar f03_limites.py y limpiar los datos temporales.

Resultado esperado: Los resultados conservan el signo y la precisión; los valores inválidos se rechazan con 422 en todos los endpoints, incluidos los módulos legados.

Evidencia: f03_limites.json

P04 — Contrato HTTP y validación Pydantic en /ai/chat

Componente: Presentación: POST /api/v1/ai/chat, AIChatRequestDTO y ai_router.py.

Procedimiento:

1. En Swagger ejecutar una solicitud válida: {"message": "Hola"}.
2. Probar mensaje ausente, mensaje vacío, solo espacios y tipo no textual.
3. Probar un campo adicional y el envío como formulario.
4. Probar Content-Type text/plain, JSON mal formado y UTF-8 inválido.
5. Repetir sin token y contrastar con POST /materials usando un precio inválido.
6. Ejecutar f04_contrato.py.

Resultado esperado: Solicitud válida 200; errores de esquema 422; tipo de contenido 415; cuerpo inválido 400; sin token 401. El endpoint legado /materials responde igual (422) tras la corrección.

Evidencia: f04_contrato.json

P05 — Interpretación del mensaje y extracción de cantidades

Componente: Aplicación: app/application/ai_use_cases.py (reglas léxicas y expresión regular de cantidades).

Procedimiento:

1. Enviar «Cotiza 2 kg de glicerina» y sus variantes en mayúsculas y sin espacio («2kg»).
2. Probar «Necesito 2 kg de glicerina», «Cotiza 2 kilos de glicerina» y «Cotiza dos kg de glicerina».
3. Probar dos ítems en una frase y un material inexistente.
4. Probar una negación («No tengo problemas con mi pedido») y una consulta con término técnico.
5. Ejecutar f05_prompt.py y comparar esperado contra real.

Resultado esperado: Las diez variantes se resuelven correctamente: números en palabras, unidades alternativas, negaciones que no deben transferir y términos técnicos que no anulan la cotización.

Evidencia: f05_prompt.json

P06 — Recuperación y aplicación de reglas RAG

Componente: Infraestructura: mock_vector_db.py; aplicación: ai_use_cases.py.

Procedimiento:

1. Inspeccionar buscar_contexto() con mensajes que contengan «IVA», «impuesto», «tope», «stock» y sin palabras clave.
2. Desde Swagger, cotizar 2 kg de glicerina con IVA y verificar 36.000 + 6.840 = 42.840.
3. Cotizar 1.000 kg de una materia prima de alto valor con «tope máximo» y verificar el recorte a 50.000.000.
4. Probar «al máximo» sin pedir el tope y registrar si la regla se aplica igualmente.
5. Ejecutar f06_rag.py.

Resultado esperado: El adaptador recupera el contexto esperado y el caso de uso aplica la regla correspondiente. El tope solo se activa con términos explícitos («tope», «límite», «no superar»), no con «al máximo».

Evidencia: f06_rag.json

P07 — Adaptador MCP: contrato y transporte real

Componente: Infraestructura: app/infrastructure/mcp_server.py; pruebas tests/test_mcp_adapter.py.

Procedimiento:

1. Ejecutar «pytest -q tests/test_mcp_adapter.py».
2. Listar las herramientas y revisar el esquema de calcular_pedido: items requerido, aplicar_iva y aplicar_tope opcionales.
3. Invocar listar_materiales, consultar_stock y calcular_pedido.
4. Comprobar que una herramienta inexistente y un pedido sin stock producen error controlado.
5. Ejecutar f07_mcp.py, que además abre una sesión MCP completa por stdio (initialize → list_tools → call_tool).

Resultado esperado: El adaptador delega en el dominio, respeta el contrato y funciona por transporte real. Estas pruebas sí demuestran una sesión MCP completa, a diferencia de una llamada directa al método.

Evidencia: f07_mcp.json · tests/test_mcp_adapter.py

P08 — Integración de extremo a extremo

Componente: Todas las capas: presentación → aplicación → RAG/MCP → dominio → SQLite.

Procedimiento:

1. Ejecutar «pytest -q tests/test_ai_presentation.py».
2. Desde Swagger, enviar una cotización con IVA y revisar status y contenido.
3. Crear un pedido de 2 kg, verificar el descuento de stock y el registro en audit-logs.
4. Cancelar el pedido, comprobar la restauración del stock y el nuevo registro de auditoría.
5. Ejecutar f08_e2e.py.

Resultado esperado: La API responde de forma coherente: 200 en la cotización, 201 al crear, stock descontado y restaurado, y ambos eventos trazados en auditoría.

Evidencia: f08_e2e.json

P09 — Manejo de errores y robustez

Componente: Presentación y aplicación: ai_router.py, ai_use_cases.py y routers legados.

Procedimiento:

1. Probar mensaje no reconocido, mensaje vacío, JSON mal formado y Content-Type inválido.
2. Probar campos adicionales y revisar que no alteren el comportamiento.
3. Probar un pedido inválido y una materia prima inválida en los endpoints legados.
4. Revisar que las respuestas no expongan trazas, nombres de archivos ni paquetes internos.
5. Ejecutar f09_errores.py.

Resultado esperado: Los errores se comunican con 4xx y sin filtrar información interna; no quedan respuestas 500 en ninguno de los endpoints auditados.

Evidencia: f09_errores.json

P10 — Regresión y consistencia de la arquitectura

Componente: Todas las capas y la suite completa de pruebas.

Procedimiento:

1. Ejecutar «pytest -q» y guardar la salida.
2. Relacionar cada archivo de prueba con la capa que verifica.
3. Analizar los imports del dominio y de la aplicación: no deben depender de FastAPI, SQLAlchemy ni MCP.
4. Registrar los fallos preexistentes y repetir las pruebas tras cualquier corrección.
5. Ejecutar f10_regresion.py.

Resultado esperado: Suite reproducible, responsabilidades por capa identificadas y defectos trazables. Los dos fallos preexistentes (test_auth y test_order_permissions) se documentan sin confundirlos con regresiones del laboratorio.

Evidencia: f10_regresion.json · f10_pytest.txt

P11 — Motor de lenguaje y respaldo por reglas

Componente: Aplicación: ai_use_cases.py; infraestructura: deepseek_llm.py (API compatible con OpenAI).

Procedimiento:

1. Sin DEEPSEEK_API_KEY, ejecutar f11_deepseek.py y comprobar que el campo «motor» es «reglas».
2. Copiar .env.example a .env, completar la clave y reiniciar el backend.
3. Repetir la ejecución y comprobar que «motor» corresponde al modo configurado («deepseek» o «deepseek-conversacional») y medir la latencia.
4. Forzar un fallo (clave inválida o sin red) y verificar que la respuesta sigue llegando por reglas.

Resultado esperado: El asistente responde con el motor configurado, informa cuál utilizó y cae a reglas ante cualquier fallo del modelo, sin exponer la clave ni trazas.

Evidencia: f11_deepseek.json

P12 — Modo conversacional y memoria de la conversación

Componente: Aplicación: ai_use_cases.py; infraestructura: deepseek_llm.py; configuración: AI_MODE.

Procedimiento:

1. Levantar una instancia con AI_MODE=conversacional (f12 lo hace en el puerto 8001).
2. Enviar un saludo y comprobar que responde de forma natural o con la plantilla de respaldo.
3. Preguntar «¿Qué venden?» y verificar que lista materiales reales del catálogo.
4. Enviar una cotización y verificar que el campo «datos» trae subtotal, IVA y total calculados.
5. Enviar un seguimiento con historial («¿Y cuánto hay de ese material?») y revisar que el contrato lo acepta y que el contexto se usa.
6. Ejecutar f12_conversacional.py con y sin clave.

Resultado esperado: El modo conversacional responde como asesor, lista el catálogo real, mantiene el contrato, devuelve cifras estructuradas y cae a la plantilla si el modelo no está disponible.

Evidencia: f12_conversacional.json

9. Matriz resumida de pruebas

ID	Prueba	Capa principal	Evidencia
P01	Operaciones y cálculo de pedido	Dominio	Salida pytest / f01
P02	Stock y consistencia transaccional	Dominio/Aplicación	f02_stock.json
P03	Negativos y decimales	Dominio/Presentación	f03_limites.json
P04	Contrato HTTP/Pydantic	Presentación	f04_contrato.json
P05	Prompt y extracción de cantidades	Aplicación	f05_prompt.json
P06	Reglas RAG	Infraestructura/Aplicación	f06_rag.json
P07	Adaptador MCP y transporte	Infraestructura	f07_mcp.json
P08	Flujo extremo a extremo	Todas	f08_e2e.json
P09	Errores y robustez	Presentación/Aplicación	f09_errores.json
P10	Regresión y arquitectura	Todas	f10_regresion.json
P11	Motor DeepSeek y respaldo	Aplicación/Infraestructura	f11_deepseek.json
P12	Modo conversacional	Aplicación/Infraestructura	f12_conversacional.json

10. Entregables

- Informe en PDF y DOCX con los hallazgos, su severidad y las correcciones propuestas.
- Carpeta docs/laboratorio4 con los scripts f01–f12 y la carpeta evidencias.
- Salida completa de pytest (f10_pytest.txt) y los JSON crudos de cada caso.
- Enlace al repositorio, rama lab4-mcp-rag, sin datos personales ni tokens.
- Tabla de adaptación respecto a la guía original (Anexo A).

11. Criterios de evaluación

Criterio	Peso	Descripción
Ejecución de los casos	30%	Los diez casos se ejecutan y se documentan.
Evidencias	25%	JSON, consolas y capturas verificables y sanitizadas.
Hallazgos	20%	Defectos identificados con severidad y justificación.
Correcciones propuestas	15%	Propuestas viables y trazables a cada hallazgo.
Arquitectura	10%	Análisis de capas, puertos y adaptadores.

Anexo A. Correspondencia con la guía original

Guía original (calculadora)	Adaptación en Kohem Chemicals
Cuatro operaciones básicas	PricingDomainService: subtotal, total, IVA y tope
División por cero	Stock insuficiente y consistencia transaccional
calculator_use_case.py	ai_use_cases.AIAssistantUseCase
mock_vector_db con impuesto 15% y tope 1000	IVA 19%, tope 50.000.000 COP y stock mínimo 10 kg
mcp_server con la herramienta calcular	Kohem-Chemicals-MCP: listar_materiales, consultar_stock, calcular_pedido
Componente LLM simulado	Reglas por defecto y DeepSeek opcional (deepseek_llm.py)
Conversación de un solo turno	Modo conversacional con historial y redacción del modelo
POST /api/v1/calcular	POST /api/v1/ai/chat
Puerto 7000	Puerto 8000
Respuesta CalculoResult	AIChatResponseDTO (response, transfer_to_employee, reason, motor)

Anexo B. Resultados de la ejecución de referencia

Valores obtenidos el 10/11/2026 sobre la rama lab4-conversacional. La evidencia completa está en docs/laboratorio4/evidencias.

Suite completa: 74 passed, 3 warnings in 3.35s.

ID	Caso	Resultado observado
P01	Operaciones y cálculo	Cumple
P02	Stock y atomicidad	Cumple
P03	Límites y precisión	Cumple
P04	Contrato HTTP	Cumple
P05	Interpretación	10/10 prompts
P06	Reglas RAG	Cumple
P07	Adaptador MCP	Cumple
P08	Extremo a extremo	Cumple
P09	Errores	Sin HTTP 500
P10	Regresión y arquitectura	Cumple
P11	Motor DeepSeek / reglas	Cumple
P12	Modo conversacional	Cumple